Lecture 14: Data II

> 第 14 讲：数据 II

Last lecture:

> 上一讲：

- Live service (e.g., GitHub) → dump/crawl (e.g., GitHub Archive) → processed data (e.g., The Stack)

> 在线服务（如 GitHub）→ 导出/爬取（如 GitHub Archive）→ 处理后的数据（如 The Stack）

- Considerations: terms of service, copyright (licenses or fair use)

> 注意事项：服务条款、版权（许可证或合理使用）

This lecture:

> 本讲内容：

- Data pipeline: transformation, filtering, deduplication, mixing

> 数据流水线：转换、过滤、去重、配比

- Mid-training + SFT: synthetic data

> Mid-training + SFT：合成数据

Summary:

> 小结：

- Filtering: train classifier (language id, quality, toxicity) for what good looks like

> 过滤：训练分类器（语言识别、质量、毒性）来刻画“好数据”长什么样

- Deduplication: hashing scales to large datasets for fuzzy matching

> 去重：哈希可以扩展到超大规模数据集，用于模糊匹配

- Mixing: try mixtures at small scale, extrapolate to optimal mixture and large scale

> 配比：在小规模上尝试各种混合方案，再外推到最优配比与大规模

- Applications: language identification, quality filtering, toxicity filtering

> 应用：语言识别、质量过滤、毒性过滤

- Post-training data: looks like evaluations, use of synthetic data

> 后训练数据：形式上类似评测，以及合成数据的使用

- A lot of data work is domain-specific, looking at examples, etc.

> 很多数据工作是特定领域的，需要查看具体样本等

Raw data does not come as text.

> 原始数据并不是以文本形式存在的。

It is HTML, PDF (arxiv), or directories (code repositories).

> 它是 HTML、PDF（arxiv）或目录（代码仓库）。

HTML to text (main one):

> HTML 转文本（主要的一种）：

- Remove boilerplate (e.g., navigation, ads) and extract content

> 去除样板内容（如导航、广告），提取正文

- What about images, tables, etc.?

> 图片、表格等怎么办?

- Inherently lossy (need to linearize)

> 本质上是有损的（需要线性化）

- Tools (rule-based): trafilatura, resiliparse, jusText, lynx, etc.

> 工具（基于规则）：trafilatura、resiliparse、jusText、lynx 等

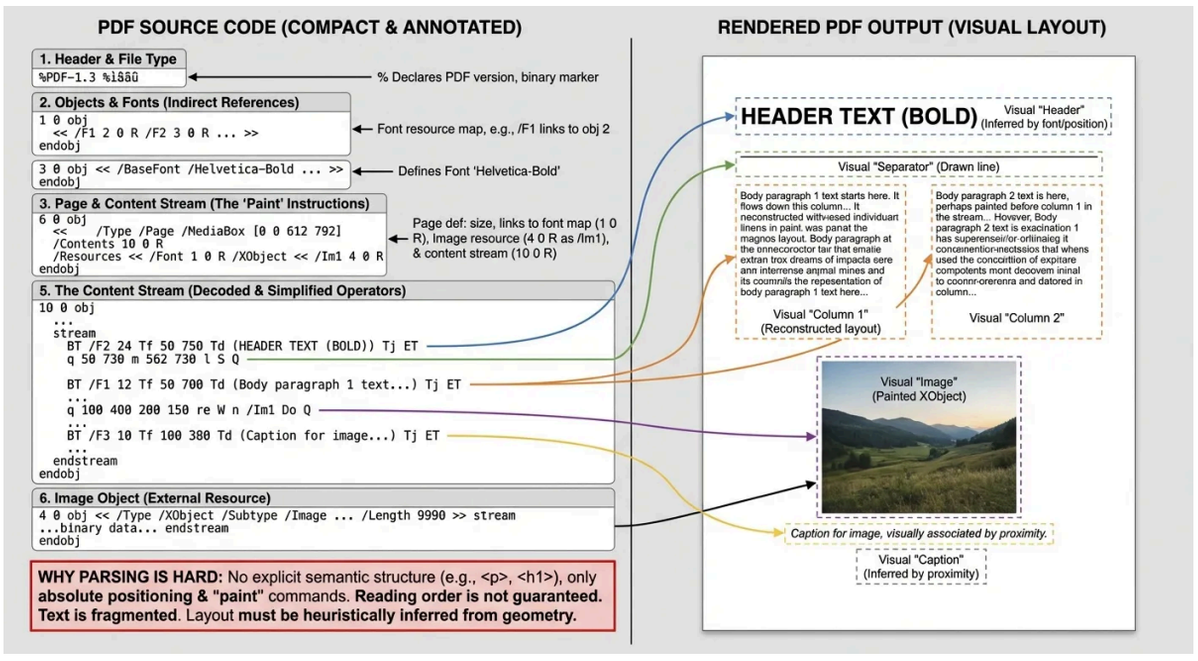
- Accuracy matters: [Li+ 2024]

> 准确性很重要： [Li+ 2024]

Text Extraction	CORE	EXTENDED
`resiliparse`	24.1	**13.4**
`trafilatura`	**24.5**	12.5
WET files	20.7	12.2

FinePDFs [post]

> FinePDFs [博文]



- Source: Common Crawl

> 来源：Common Crawl

- Recrawl truncated PDFs (since they are big)

> 重新爬取被截断的 PDF（因为它们体积很大）

- OCR (RolmOCR) using a VLM or Docling (make these run fast)

> 用 VLM 做 OCR（RolmOCR），或使用 Docling（要让它们跑得快）

- Lots of cleanup and filtering

> 大量清洗与过滤

- A lot of layout information is missing

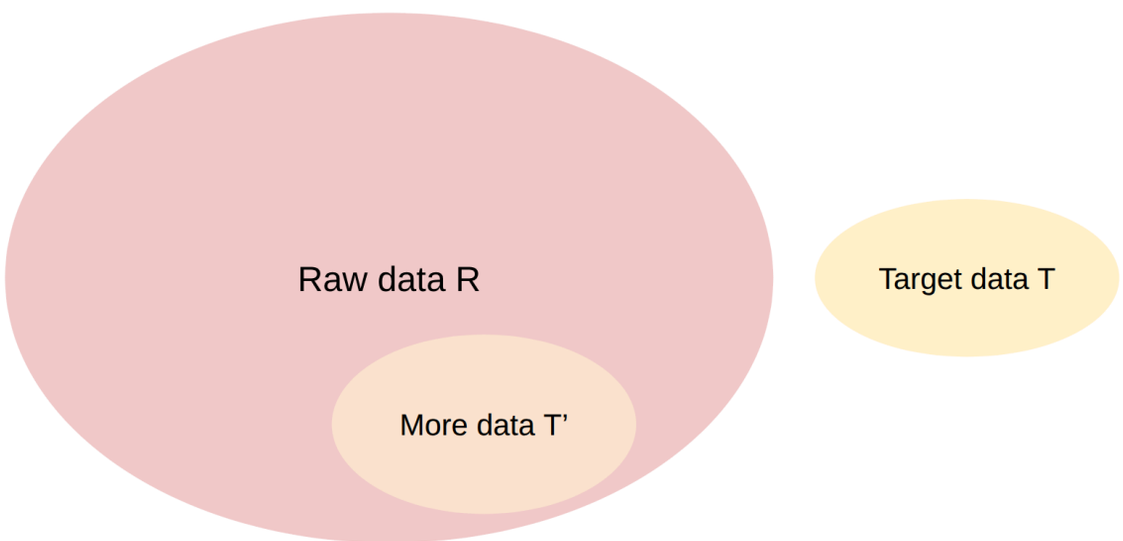
> 大量版面信息丢失

Algorithmic building block:

> 算法构建块：

- Given some **target data** T and lots of **raw data** R, find subset T' of R similar to T.

> 给定目标数据 T 和大量原始数据 R，找到 R 中与 T 相似的子集 T'。



Applications:

> 应用：

- Language identification (English versus rest)

> 语言识别（英语 vs 其他语言）

- Quality filtering (high quality versus low quality)

> 质量过滤（高质量 vs 低质量）

- Toxicity filtering (non-toxic versus toxic)

> 毒性过滤（无毒 vs 有毒）

Desiderata for filtering algorithm:

> 对过滤算法的期望：

- Generalize from the target data (want T and T' to be different)

> 能从目标数据泛化（希望 T 与 T' 不同）

- Extremely fast (have to run it on R, which is huge)

> 极其快速（必须在庞大的 R 上运行它）

Survey paper on data selection [Albalak+ 2024]

> 关于数据选择的综述论文 [Albalak+ 2024]

General framework: Given target T and raw R, find subset of R similar to T

> 通用框架：给定目标 T 与原始数据 R，找到 R 中与 T 相似的子集

1. Estimate some model based on R and T and derive a scoring function

> 基于 R 和 T 估计某个模型，并导出打分函数

2. Keep examples in R based on their score

> 根据得分保留 R 中的样本

Types of classifiers:

> 分类器的类型：

- Generative model of T (KenLM): score(x) = p_T(x)

> T 的生成模型（KenLM）：score(x) = p_T(x)

- Simple classifier (fastText): score(x) = p(T | x)

> 简单分类器（fastText）：score(x) = p(T | x)

To use: keep examples x with score(x) >= threshold (stochastically)

> 用法：保留 score(x) >= 阈值的样本 x（按概率随机保留）

Model-based filtering?

> 要不要用基于模型的过滤?

- Some deliberately do not use model-based filtering (C4, Gopher, RefinedWeb, FineWeb, Dolma)

> 有些工作刻意不使用基于模型的过滤（C4、Gopher、RefinedWeb、FineWeb、Dolma）

- Some use model-based filtering (GPT-3, LLaMA, DCLM) [becoming the norm]

> 有些工作则使用基于模型的过滤（GPT-3、LLaMA、DCLM）[正成为常态]

Language identification:

> 语言识别：

- Goal: find text of a specific language (e.g., English)

> 目标：找到特定语言的文本（如英语）

- fastText language identification [article]

> fastText 语言识别 [文章]

- Off-the-shelf classifier

> 现成的分类器

- Supports 176 languages

> 支持 176 种语言

- Trained on multilingual sites: Wikipedia, Tatoeba (translation site) and SETimes (Southeast European news)

> 在多语言网站上训练：Wikipedia、Tatoeba（翻译网站）和 SETimes（东南欧新闻）

- Dolma keeps pages with p(English) >= 0.5 [Soldaini+ 2024]

> Dolma 保留 p(English) >= 0.5 的页面 [Soldaini+ 2024]

OpenMathText [Paster+ 2023]

- Goal: curate large corpus of mathematical text from CommonCrawl

> 目标：从 CommonCrawl 中整理出大规模数学文本语料

- Use rules to filter (e.g., contains latex commands)

> 用规则做过滤（如包含 latex 命令）

- KenLM trained on ProofPile, keep if perplexity < 15000

> KenLM 在 ProofPile 上训练，困惑度 < 15000 则保留

- Trained fastText classifier to predict mathematical writing, threshold is 0.17 if math, 0.8 if no math

> 训练 fastText 分类器来预测数学写作，阈值：判定为数学取 0.17，判定为非数学取 0.8

- Result: produced 14.7B tokens, used to train 1.4B models that do better than models trained on 20x data

> 结果：产出 14.7B tokens，用它训练的 1.4B 模型优于在 20 倍数据上训练出的模型

GPT-3 [Brown+ 2020]

- Positives: samples from {Wikipedia, WebText2, Books1, Books2}

> 正例：从 {Wikipedia, WebText2, Books1, Books2} 中采样

- Negatives: samples from CommonCrawl

> 负例：从 CommonCrawl 采样

Train linear classifier based on word features [article]

> 基于词特征训练线性分类器 [文章]

Keep documents stochastically based on score

> 根据得分按概率随机保留文档

LLaMA/RedPajama [Touvron+ 2023]

- Positives: samples from pages **referenced** by Wikipedia

> 正例：从 Wikipedia 引用的页面中采样

- Negatives: samples from CommonCrawl

> 负例：从 CommonCrawl 采样

- Keep documents that are classified positive

> 保留被分类为正例的文档

phi-1 [Gunasekar+ 2023]

- Philosophy: really high quality data (textbooks) to train a small model (1.5B)

> 理念：用真正高质量的数据（教科书）训练小模型（1.5B）

- Includes synthetic data from GPT 3.5 (later: GPT-4) and filtered data

> 包含来自 GPT 3.5（后来是 GPT-4）的合成数据以及过滤后的数据

- Train random forest classifier on T using output embedding from pretrained codegen model

> 使用预训练 codegen 模型的输出嵌入，在 T 上训练随机森林分类器

- Select data from R that is classified positive by the classifier

> 从 R 中选取被该分类器判为正例的数据

Result on HumanEval:

> 在 HumanEval 上的结果：

- Train 1.3B LM on Python subset of The Stack (performance: 12.19% after 96K steps)

> 在 The Stack 的 Python 子集上训练 1.3B LM（性能：96K 步后 12.19%）

- Train 1.3B LM on new filtered subset (performance: 17.68% after 36K steps) - better!

> 在新的过滤子集上训练 1.3B LM（性能：36K 步后 17.68%）——更好！

Toxicity filtering in Dolma [Soldaini+ 2024]

> Dolma 中的毒性过滤 [Soldaini+ 2024]

- Dataset: Jigsaw Toxic Comments dataset (2018) [dataset]

> 数据集：Jigsaw Toxic Comments 数据集（2018） [数据集]

- Project goal: help people have better discussions online [article]

> 项目目标：帮助人们在网络上进行更好的讨论 [文章]

- Data: comments on Wikipedia talk page annotated with {toxic, severe_toxic, obscene, threat, insult, identity_hate}

> 数据：Wikipedia 讨论页上的评论，标注有 {toxic, severe_toxic, obscene, threat, insult, identity_hate}

Scale-dependent effects of filtering:

> 过滤的规模相关效应：

- No single optimal threshold for filtering

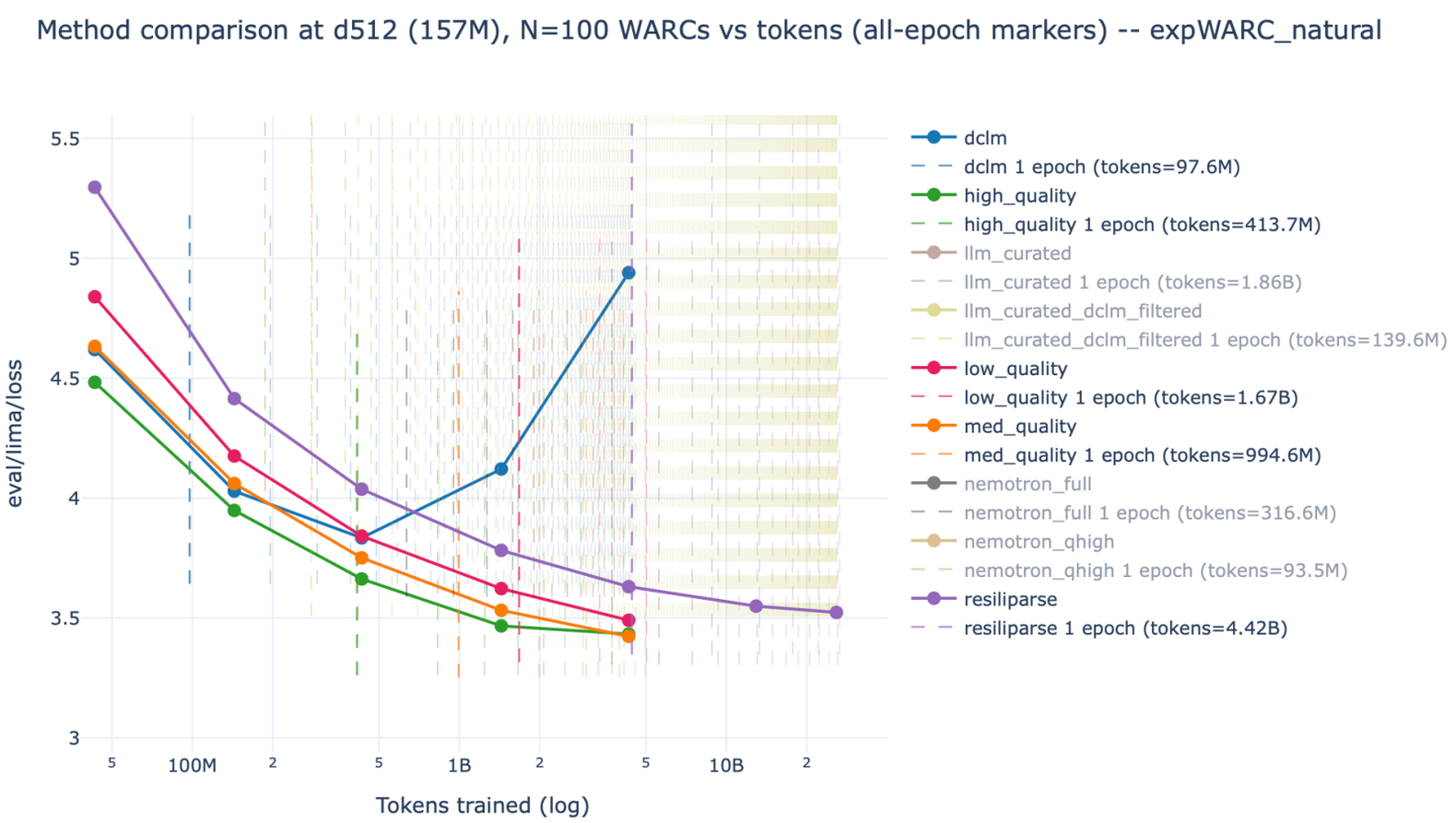
> 过滤不存在单一的最优阈值

- If training for longer, want more (lower quality) data

> 训练时间更长时，想要更多（质量更低的）数据

- If training for shorter, want less (higher quality) data

> 训练时间更短时，想要更少（质量更高的）数据



Summary:

> 小结：

- Filtering is critical for building a good model

> 过滤对构建好模型至关重要

- Recipe: define target data (what good looks like), extrapolate to raw data

> 配方：定义目标数据（好数据长什么样），再外推到原始数据

Two types of duplicates:

> 两类重复：

- Exact duplicates (mirror sites, GitHub forks) [Gutenberg mirrors]

> 完全重复（镜像站点、GitHub forks） [Gutenberg 镜像]

- Near duplicates: same text differing by a few tokens

> 近似重复：文本相同，仅相差少数 token

Examples of near duplicates:

> 近似重复的例子：

- Terms of service and licenses [MIT license]

> 服务条款与许可证 [MIT 许可证]

- Formulaic writing (copy/pasted or generated from a template)

Dataset	Example	Near-Duplicate Example
Wiki-40B	\n_START_ARTICLE_\nHum Award for Most Impactful Character \n_START_SECTION_\nWinners and nominees\n_START_PARAGRAPH_\nIn the list below, winners are listed first in the colored row, followed by the other nominees. [...]	\n_START_ARTICLE_\nHum Award for Best Actor in a Negative Role \n_START_SECTION_\nWinners and nominees\n_START_PARAGRAPH_\nIn the list below, winners are listed first in the colored row, followed by the other nominees. [...]
LM1B	I left for California in 1979 and tracked Cleveland 's changes on trips back to visit my sisters .	I left for California in 1979 , and tracked Cleveland 's changes on trips back to visit my sisters .
C4	Affordable and convenient holiday flights take off from your departure country, "Canada". From May 2019 to October 2019, Condor flights to your dream destination will be roughly 6 a week! Book your Halifax (YHZ) - Basel (BSL) flight now, and look forward to your "Switzerland" destination!	Affordable and convenient holiday flights take off from your departure country, "USA". From April 2019 to October 2019, Condor flights to your dream destination will be roughly 7 a week! Book your Maui Kahului (OGG) - Dubrovnik (DBV) flight now, and look forward to your "Croatia" destination!

> 模板化写作（复制粘贴或由模板生成）

- Minor formatting differences in copy/pasting

> 复制粘贴造成的细微格式差异

Product description repeated 61,036 times in C4

> 一条商品描述在 C4 中重复了 61,036 次

'"by combining fantastic ideas, interesting arrangements, and follow the current trends in the field of that make you more inspired and give artistic touches. We'd be honored if you can apply some or all of these design in your wedding. believe me, brilliant ideas would be perfect if it can be applied in real and make the people around you amazed!

> "通过融合绝妙的创意、有趣的布置，并紧跟该领域当前的潮流，让你获得更多灵感、增添艺术气息。如果您能将这些设计中的一部分或全部应用到您的婚礼中，我们将深感荣幸。相信我，绝妙的想法若能付诸现实、让您身边的人为之惊叹，那就再完美不过了！

[example page]

> [示例页面]

Deduplication training data makes language models better [Lee+ 2021]

> 对训练数据去重能让语言模型更好 [Lee+ 2021]

- Train more efficiently (because have fewer tokens)

> 训练更高效（因为 token 更少）

- Avoid memorization (can mitigate copyright, privacy concerns)

> 避免记忆化（可以缓解版权、隐私方面的担忧）

Design space:

> 设计空间：

1. What is an item (sentence, paragraph, document)?

> 什么算一个条目（句子、段落、文档）？

2. How to match (exact match, existence of common subitem, fraction of common subitems)?

> 如何匹配（精确匹配、存在共同子条目、共同子条目占比）？

3. What action to take (remove all, remove all but one)?

> 采取什么动作（全部删除、保留一个其余删除）？

Key challenge:

> 关键挑战：

- Deduplication is fundamentally about comparing items to other items

> 去重从根本上说就是把条目与其他条目做比较

- Need linear time algorithms to scale

> 需要线性时间算法才能扩展

- Hash function h maps item to a hash value (integer or string)

> 哈希函数 h 将条目映射为一个哈希值（整数或字符串）

- Hash value much smaller than item

> 哈希值远小于条目本身

- Hash collision: h(x) = h(y) for x ≠ y

> 哈希碰撞：当 x ≠ y 时 h(x) = h(y)

Tradeoff between efficiency and collision resistance [article]

> 效率与抗碰撞性之间的权衡 [文章]

- Cryptographic hash functions (SHA-256): collision resistant, slow (used in bitcoin)

> 密码学哈希函数（SHA-256）：抗碰撞，但慢（比特币中使用）

- DJB2, MurmurHash, CityHash: not collision resistant, fast (used for hash tables)

> DJB2、MurmurHash、CityHash：不抗碰撞，但快（用于哈希表）

We will use MurmurHash:

> 我们将使用 MurmurHash：

Simple example

> 简单示例

1. Item: string

> 条目：字符串

2. How to match: exact match

> 如何匹配：精确匹配

3. Action: remove all but one

> 动作：保留一个，其余删除

- Pro: simple, clear semantics, high precision

> 优点：简单、语义清晰、精确率高

- Con: does not deduplicate near duplicates

> 缺点：无法去除近似重复

- This code is written in a MapReduce way, can easily parallelize and scale

> 这段代码以 MapReduce 的方式编写，可以轻松并行化与扩展

C4 [Raffel+ 2019]

1. Item: 3-sentence spans

> 条目：3 句组成的片段

2. How to match: use exact match

> 如何匹配：使用精确匹配

3. Action: remove all but one

> 动作：保留一个，其余删除

Warning: when a 3-sentence span is removed from the middle of a document, the resulting document might not be coherent

> 警告：当从文档中间删除一个 3 句片段时，得到的文档可能不再连贯

Let's now look at approximate set membership.

> 现在来看近似集合成员测试。

First we need a similarity measure.

> 首先我们需要一个相似度度量。

Jaccard similarity

> Jaccard 相似度

Definition: Jaccard(A, B) = |A intersect B| / |A union B|

> 定义：Jaccard(A, B) = |A intersect B| / |A union B|

Definition: two documents are **near duplicates** if their Jaccard similarity >= threshold

> 定义：若两个文档的 Jaccard 相似度 >= 阈值，则它们是近似重复

Algorithmic challenge: find near duplicates in linear time

> 算法挑战：在线性时间内找出近似重复

MinHash

> MinHash

MinHash: a random hash function h so that Pr[h(A) = h(B)] = Jaccard(A, B)

> MinHash：一个随机哈希函数 h，使得 Pr[h(A) = h(B)] = Jaccard(A, B)

Normally, you want different items to hash to different hashes

> 通常，你希望不同的条目哈希到不同的哈希值

...but here, you want collision probability to depend on similarity

> ……但在这里，你希望碰撞概率取决于相似度

Characteristic matrix representation:

> 特征矩阵表示：

```
item | A | B
```

> 条目 | A | B

```
1    | 1 | 1
2    | 1 | 1
3    | 1 | 1
4    | 1 | 0
5    | 0 | 1
```

Random hash function induces a permutation over items

> 随机哈希函数在条目上诱导出一个排列

Look at which item is first in A and which item is first in B.

> 看哪个条目在 A 中排第一、哪个条目在 B 中排第一。

Each item has the same probability as being first (min)

> 每个条目成为第一个（min）的概率相同

- If 1, 2, 3 is first, then first in A = first in B.

> 若排第一的是 1、2、3，则 A 中的第一个 = B 中的第一个。

- If 4, 5 is first, then first in A ≠ first in B.

> 若排第一的是 4、5，则 A 中的第一个 ≠ B 中的第一个。

Now we can hash our items, but a collision doesn't tell us Jaccard(A, B) > threshold.

> 现在我们可以对条目做哈希，但发生一次碰撞并不能告诉我们 Jaccard(A, B) > 阈值。

Locality sensitive hashing (LSH) [book chapter]

> 局部敏感哈希（LSH）[书籍章节]

Suppose we hash examples with just one MinHash function

> 假设只用一个 MinHash 函数对样本做哈希

P[A and B collide] = Jaccard(A, B)
On average, more similar items will collide, but very stochastic...

> 平均而言，越相似的条目越容易碰撞，但随机性非常强……

Goal: have A and B collide if Jaccard(A, B) > threshold

> 目标：当 Jaccard(A, B) > 阈值时，让 A 和 B 碰撞

We have to somehow sharpen the probabilities...

> 我们得以某种方式锐化这些概率……

Solution: use n hash functions

> 解决方案：使用 n 个哈希函数

Break up into b bands of r hash functions each (n = b * r)

> 划分成 b 个 band，每个 band 含 r 个哈希函数（n = b * r）

Hash functions:

> 哈希函数：

```
h1 h2 h3 h4  |  h5 h6 h7 h8  |  h9 h10 h11 h12
```

Key: A and B collide if for *some* band, *all* its hash functions return same value

> 关键：若存在某个 band，其所有哈希函数都返回相同的值，则 A 和 B 碰撞

As we will see, the and-or structure of the bands sharpens the threshold

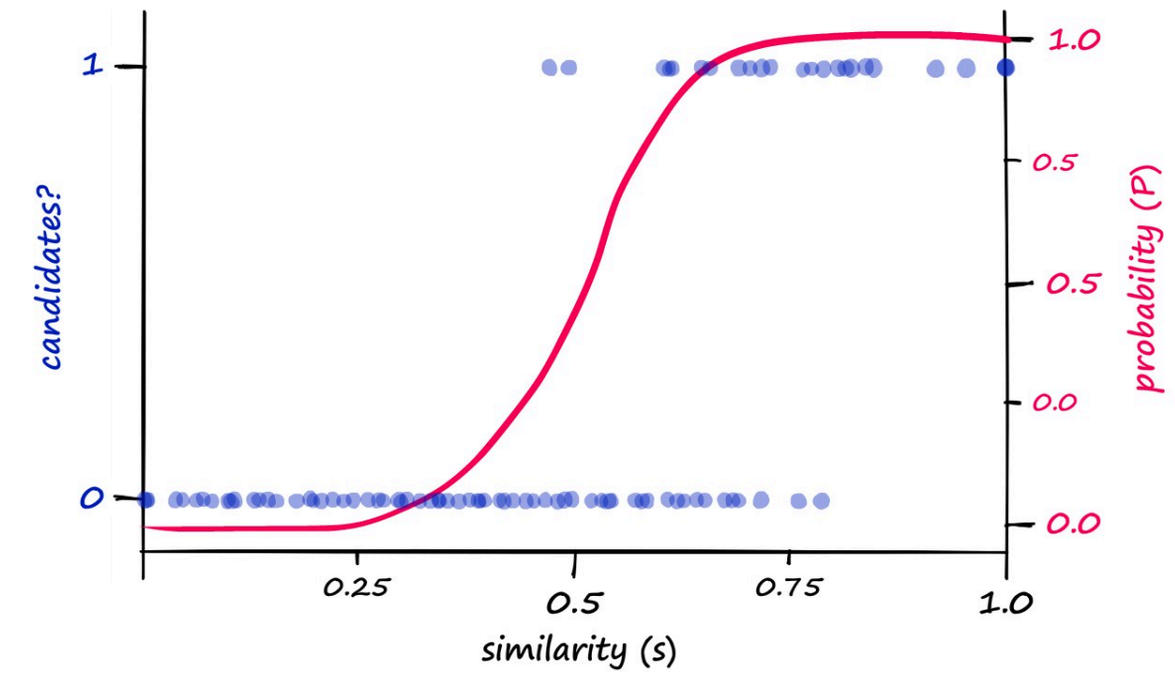
> 后面会看到，band 的 and-or 结构会锐化阈值

Given Jaccard(A, B), what is the probability that A and B collide?

> 给定 Jaccard(A, B)，A 和 B 碰撞的概率是多少?

Example

> 示例

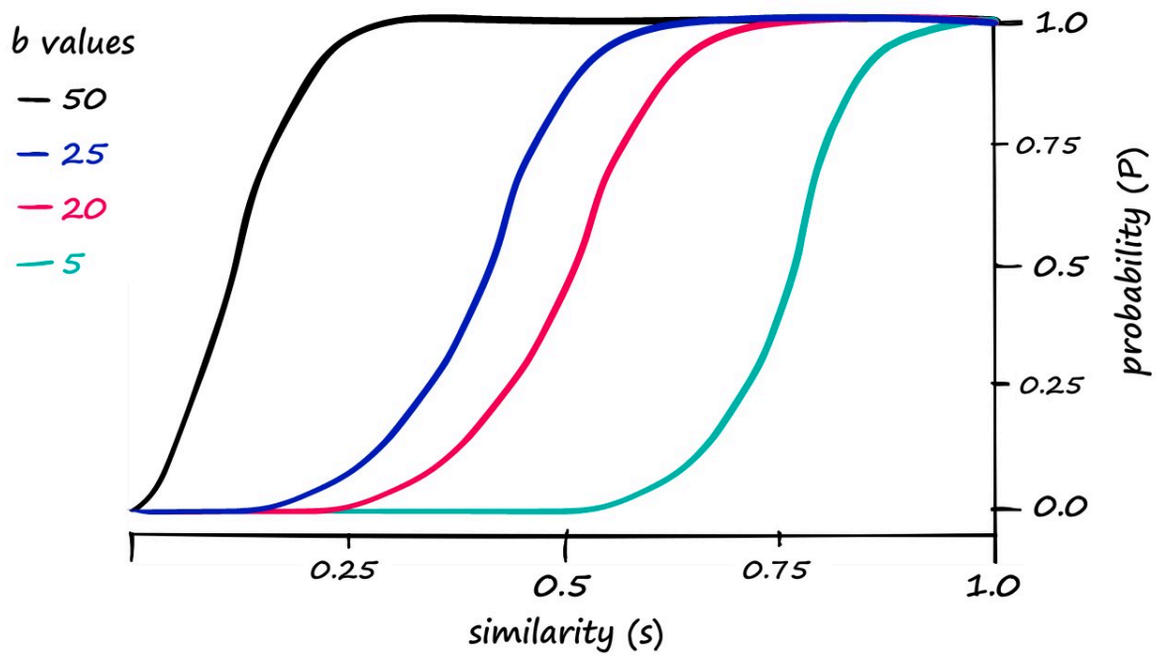


Increasing r sharpens the threshold and moves the curve to the right (harder to match)

> 增大 r 会锐化阈值并使曲线右移（更难匹配）

Increasing b moves the curve to the left (easier to match)

> 增大 b 会使曲线左移（更容易匹配）



Example setting [Lee+ 2021]: n = 9000, b = 20, r = 450

> 示例设置 [Lee+ 2021]：n = 9000，b = 20，r = 450

What is the threshold (where the phase transition happens)?

> 阈值是多少（相变发生在哪里）？

Probability that a fixed band matches:

> 某个固定 band 匹配的概率：

Probability that A and B collide is a constant (≈ 1-1/e):

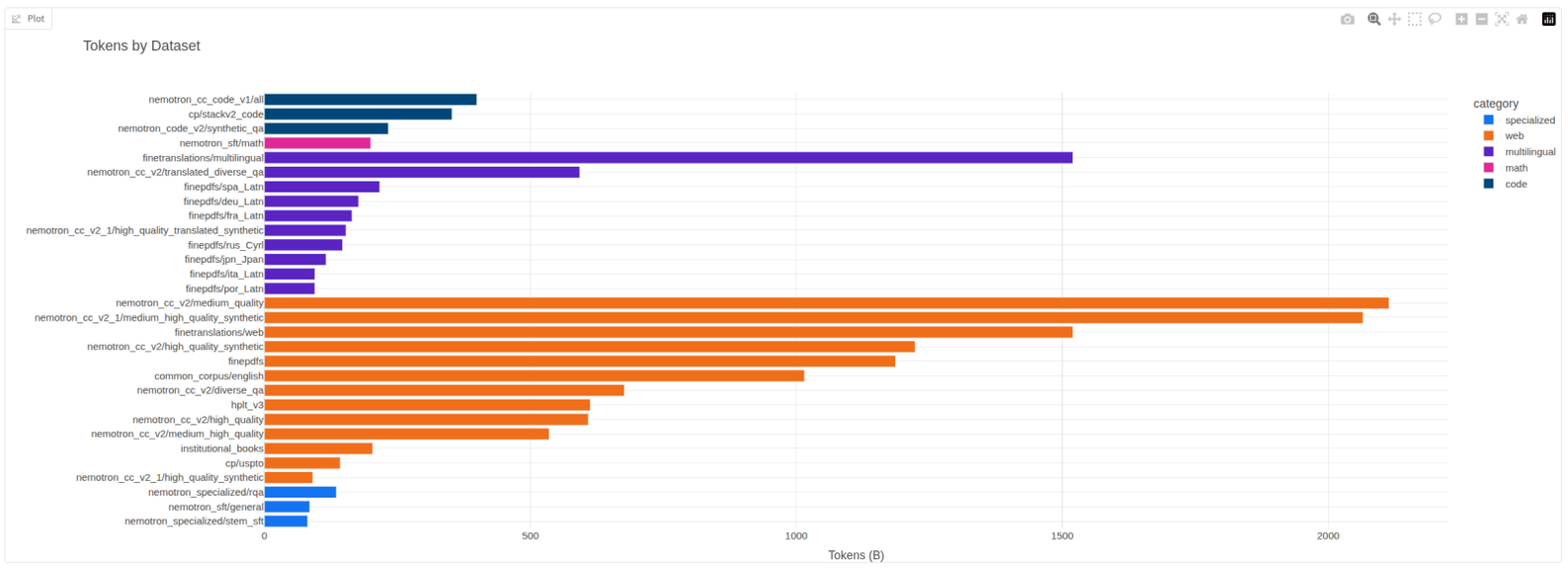
> A 和 B 碰撞的概率是一个常数（≈ 1-1/e）：

Recall that language models are trained on multiple data sources.

> 回顾：语言模型是在多个数据源上训练的。

Datasets in Marin: [token viewer]

> Marin 中的数据集：[token 查看器]



The Pile [Gao+ 2020]

Component	Raw Size	Weight	Epochs	Effective Size	Mean Document Size
Pile-CC	227.12 GiB	18.11%	1.0	227.12 GiB	4.33 KiB
PubMed Central	90.27 GiB	14.40%	2.0	180.55 GiB	30.55 KiB
Books3†	100.96 GiB	12.07%	1.5	151.44 GiB	538.36 KiB
OpenWebText2	62.77 GiB	10.01%	2.0	125.54 GiB	3.85 KiB
ArXiv	56.21 GiB	8.96%	2.0	112.42 GiB	46.61 KiB
Github	95.16 GiB	7.59%	1.0	95.16 GiB	5.25 KiB
FreeLaw	51.15 GiB	6.12%	1.5	76.73 GiB	15.06 KiB
Stack Exchange	32.20 GiB	5.13%	2.0	64.39 GiB	2.16 KiB
USPTO Backgrounds	22.90 GiB	3.65%	2.0	45.81 GiB	4.08 KiB
PubMed Abstracts	19.26 GiB	3.07%	2.0	38.53 GiB	1.30 KiB
Gutenberg (PG-19)†	10.88 GiB	2.17%	2.5	27.19 GiB	398.73 KiB
OpenSubtitles†	12.98 GiB	1.55%	1.5	19.47 GiB	30.48 KiB
Wikipedia (en)†	6.38 GiB	1.53%	3.0	19.13 GiB	1.11 KiB
DM Mathematics†	7.75 GiB	1.24%	2.0	15.49 GiB	8.00 KiB
Ubuntu IRC	5.52 GiB	0.88%	2.0	11.03 GiB	545.48 KiB
BookCorpus2	6.30 GiB	0.75%	1.5	9.45 GiB	369.87 KiB
EuroParl†	4.59 GiB	0.73%	2.0	9.17 GiB	68.87 KiB
HackerNews	3.90 GiB	0.62%	2.0	7.80 GiB	4.92 KiB
YoutubeSubtitles	3.73 GiB	0.60%	2.0	7.47 GiB	22.55 KiB
PhilPapers	2.38 GiB	0.38%	2.0	4.76 GiB	73.37 KiB
NIH ExPorter	1.89 GiB	0.30%	2.0	3.79 GiB	2.11 KiB
Enron Emails†	0.88 GiB	0.14%	2.0	1.76 GiB	1.78 KiB
The Pile	**825.18 GiB**			**1254.20 GiB**	**5.91 KiB**

Key question: what distribution over the data sources should we use?

> 关键问题：我们应该在各数据源上使用什么样的分布？

Example:

> 示例：

Baselines:

> 基线：

- Vibes: set p(s) manually based on intuition (quite common)

> 凭感觉：基于直觉手动设定 p(s)（相当常见）

- Uniform sampling: sample uniformly ($p(s) \propto 1$)

> 均匀采样：等概率地采样（$p(s) \propto 1$）

- Proportional mixing: sample proportional to the number of tokens in a source ($p(s) \propto \text{num_tokens}(s)$)

> 按比例混合：按数据源的 token 数量成比例采样（$p(s) \propto \text{num_tokens}(s)$）

Intuition: should upweight higher quality sources

> 直觉：应该给高质量的数据源更高的权重

However...

> 然而……

1. We want to ensure diversity (e.g., across incomparable sources: literature, code, papers)

> 我们希望保证多样性（例如在不可比的数据源之间：文学、代码、论文）

2. Each source is finite, so if put too much weight on a small source, then need to epoch over it

> 每个数据源都是有限的，如果在小数据源上放太多权重，就需要对它反复做多个 epoch

This last point is important and a bit subtle.

> 最后这一点很重要，也有些微妙。

Example:

> 示例：

50x epochs on high quality data...can lead to overfitting!

> 对高质量数据做 50 倍 epoch……可能导致过拟合！

UniMax [Chung+ 2023]

- Setting: balancing different languages for multilingual models

> 场景：为多语言模型均衡不同语言

- Previous work: between uniform and proportional mixing (p(s) ∝ num_tokens(s)^α for α in [0, 1])

> 先前工作：介于均匀与按比例混合之间（p(s) ∝ num_tokens(s)^α，α in [0, 1]）

- Idea: sample sources uniformly but with a hard **cap** C on number of epochs for any source

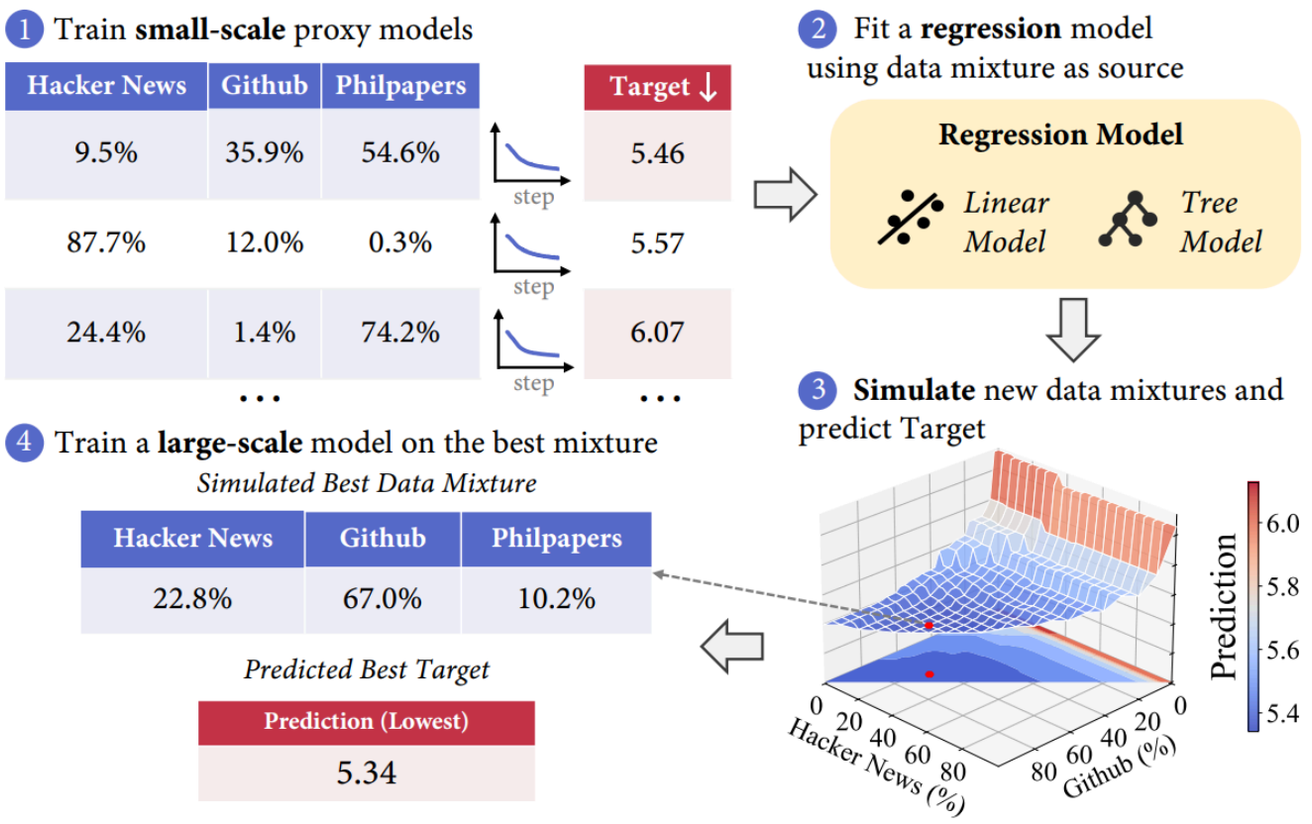
> 想法：对各数据源均匀采样，但对任意数据源的 epoch 数设置硬上限 C

- Specifically, p(s) * num_training_tokens ≤ C for all sources s

> 具体地，对所有数据源 s 都有 p(s) * num_training_tokens ≤ C

Regression-based mixing [Liu+ 2024][Chen+ 2026]

> 基于回归的配比 [Liu+ 2024][Chen+ 2026]



- Define distribution over mixtures p (e.g., Dirichlet)

> 定义混合配比上的分布 p（如 Dirichlet）

- Define regression method (e.g., linear, gradient boosted trees)

> 定义回归方法（如线性模型、梯度提升树）

- Define target based on downstream evals (careful not to overfit!)

> 基于下游评测定义目标（小心不要过拟合！）

- Discrepancy between small and large scale (tradeoff cost and accuracy)

> 小规模与大规模之间的差异（在成本与准确性之间权衡）

Design Choice	**RegMix** (Liu et al., 2025a)	**DML** (Ye et al., 2025)	**AutoScale** (Kang et al., 2025)	**BiMix** (Ge et al., 2025b)	**ADMIRE-BayesOpt** (Chen et al., 2025b)	**CLIMB** (Diao et al., 2025)	**OLMIXBASE (Algorithm 1)**
Swarm Construction							
Proxy model size	1M	70, 160, 305, 410M	Target	280M	1M, 60M	350M	30M
Swarm size (vs m domains)	512 (m = 17)	20 (m = 7)	$2m + 1$	4	101 (m = 17)	112 (m = 21)	$3(m + 1)$
Swarm distribution	Dirichlet with natural prior	Exponential grid	Exponential grid	Entropy-weighted	Dynamic	Dirichlet with natural prior	Dirichlet with natural prior (sparse for topics, dense for sources)
Regression Model							
Regression model family	LightGBM	Log-Linear	Power Law	Power Law	Gaussian Process	LightGBM	Log-Linear
Regression granularity	Aggregated	Aggregated	Per-Task	Per-Task	Aggregated	Aggregated	Per-Task
Mixture Optimization							
Data repetition constraints	No	No	No	No	No	No	Yes
Optimization solver	Search	Search	Gradient Descent	Exact Solver	Search	Search	Exact Solver with KL reg.

Hope 1: regression model is accurate at minimizer 🙏

> 希望 1：回归模型在最小值点处是准确的 🙏

Hope 2: optimal data mixtures transfer from small to large scale 🙏

> 希望 2：最优数据配比能从小规模迁移到大规模 🙏

Hold on. There's at least one scale-dependent effect:

> 等等，这里至少有一个与规模相关的效应：

- If train small models on low token counts:

> 如果在很少的 token 数上训练小模型：

- But if train large model on this mixture, we will epoch a ton on high quality data and overfit!

> 但如果用这个配比训练大模型，我们会对高质量数据做海量 epoch 并过拟合！

Simulated epoching [Held+ 2025]

> 模拟 epoch [Held+ 2025]

- General idea: make small scale look like large scale (general theme of this course)

> 总体思路：让小规模看起来像大规模（本课程的一贯主题）

- Instantiation: downsample all sources proportionally

> 具体做法：对所有数据源按比例降采样

- In this downsampled mixture, models that epoch too much won't look good.

> 在这个降采样后的配比中，epoch 过多的模型表现不会好。

- So the optimum will be more balanced.

> 因此最优解会更均衡。

Summary:

> 小结：

- Problem: how to weight different data sources (e.g., Wikipedia, general, code)

> 问题：如何为不同数据源（如 Wikipedia、通用、代码）设置权重

- Regression-based mixing: estimate mixture → loss at small scale, optimize (analogous to scaling laws)

> 基于回归的配比：在小规模上估计 配比 → 损失，再做优化（类比缩放定律）

- Important consideration: epoching and overfitting (solution: cap or simulated)

> 重要考量：多轮 epoch 与过拟合（解决方案：设上限或模拟 epoch）

Recipe:

> 配方：

1. Define a set of environments

> 定义一组环境

2. Define a set of tasks / prompts

> 定义一组任务 / 提示

3. Collect responses from a strong model (teacher)

> 从强模型（教师模型）收集回答

OpenThoughts [Guha+ 2025]

- 1.2M examples using QwQ-32B as a teacher

> 使用 QwQ-32B 作为教师生成 1.2M 个样本

- Questions come from 27 human and synthetic sources (e.g., StackExchange, NuminaMath, Chemistry)

> 问题来自 27 个人工与合成来源（如 StackExchange、NuminaMath、Chemistry）

- **StackExchange CodeGolf** (Number of Questions: 85.9K): A StackExchange forum of coding puzzles, specifically aimed at solutions with the least number of characters possible.
- **OpenCodeReasoning** (Number of Questions: 459K) A large reasoning-based synthetic dataset to date for coding, comprises 735,255 samples in Python across 28,319 unique competitive programming questions
- **cognitivecomputations/dolphin-coder** (Number of Questions: 101K): Synthetic questions evolved from LeetCode questions.
- **m-a-p/CodeFeedback-Filtered-Instruction** (Number of Questions: 150K): Mixture of synthetic and real coding questions filtered by an LLM.
- **KodCode/KodCode-V1** (Number of Questions: 384K): Fully synthetic and diverse coding dataset with questions ranging from algorithmic to package specific knowledge.
- **Multilingual-Multimodal-NLP/McEval-Instruct** (Number of Questions: 35.8K): Multilingual code dataset on code-understanding, completion, and generation.
- **christopher/rosetta-code** (Number of Questions: 75.4K): Multilingual code dataset on basic coding exercises.
- **glaiveai/glaive-code-assistant-v3** (Number of Questions: 946K): Code problems and solutions generated using Glaive's synthetic data generation platform.
- **StackExchange CodeReview** (Number of Questions: 183K): Code review questions from codereview.meta.stackexchange.com.
- **prithivMLmods/Coder-Stat** (Number of Questions: 41.9K): Coding dataset for the analysis of coding patterns, error types, and performance metrics. We transform code and an associated error into a question for the LLM to solve using GPT-4o-mini with the prompt in Figure 16.
- **OpenCoder-LLM/opc-sft-stage2**: A mixture of synthetic python questions generated from python documentation, educational material and more. We use both OpenCoder-LLM/opc-sft-stage2 and OpenCoder-LLM/opc-sft-stage1. Specifically, we use the package_instruct subset of OpenCoder-LLM/opc-sft-stage2 and the filtered_infinity_instruct, largescale_diverse_instruct, and realuser_instruct subsets of OpenCoder-LLM/opc-sft-stage1.

- Sampling multiple (16) responses per prompt is helpful

> 对每个提示采样多个（16 个）回答是有帮助的

- Better models aren't necessarily better teachers: QwQ-32B is a better teacher than DeepSeek-R1

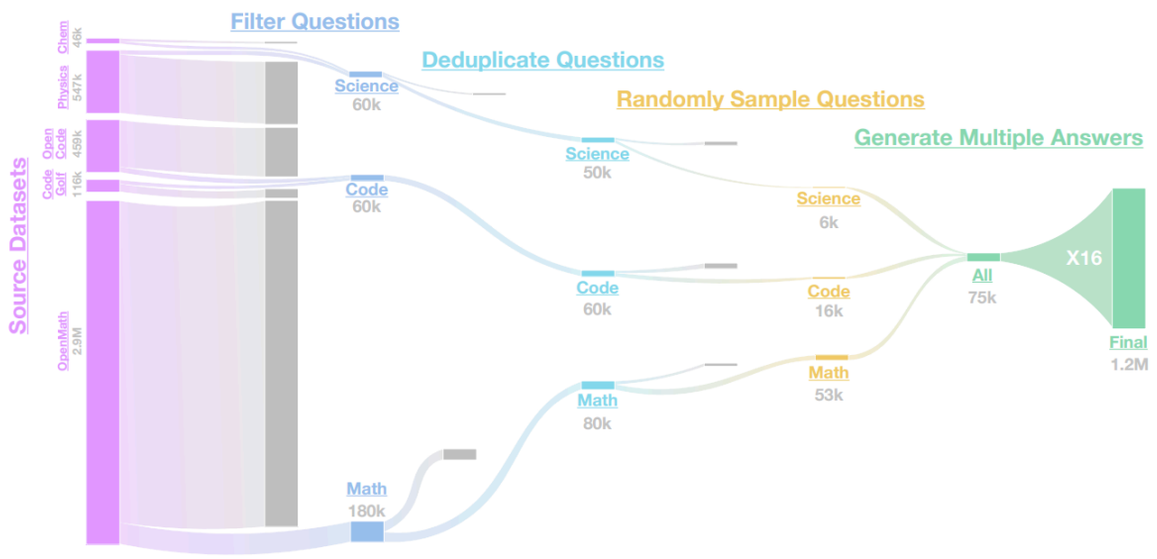
> 更好的模型未必是更好的教师：QwQ-32B 是比 DeepSeek-R1 更好的教师

- Answer filtering wasn't helpful

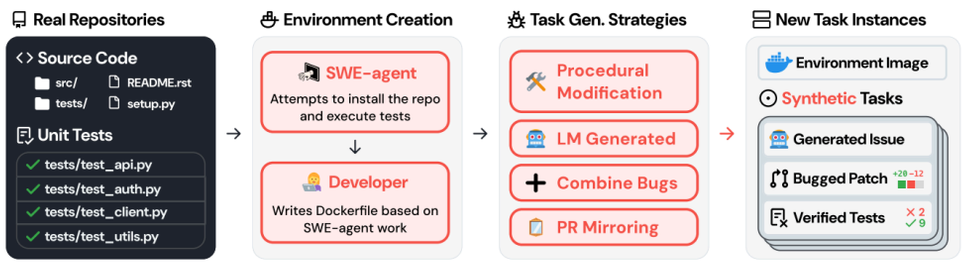
> 答案过滤没有帮助

- Smaller high quality sources (e.g., OpenMath-2-Math) is better than large diverse sources

> 较小的高质量来源（如 OpenMath-2-Math）优于庞大而多样的来源



SWE-smith [Yang+ 2025]



- Given a repository, use LM to generate tasks (introduce bugs with LM)

> 给定一个代码仓库，用 LM 生成任务（用 LM 注入 bug）

- 128 GitHub repositories yields 50K tasks

> 128 个 GitHub 仓库产出 50K 个任务

SWE-Zero[Ludwig+ 2026]

- SWE tasks have heavy dependencies (unlike math or coding contests)

> SWE 任务有很重的依赖（不像数学或编程竞赛）

- Setting up thousands of Docker images is an infrastructural nightmare

> 搭建数千个 Docker 镜像是基础设施上的噩梦

- Observation: strong models can solve many tasks without execution feedback

> 观察：强模型不需要执行反馈也能解决很多任务

Model	Execution	SWE-bench (V)	SWE-bench (M)
MiniMax-M2.5	×	69.5	57.2
	✓	80.2	74.1
Qwen3-Coder-Next	×	56.9	50.7
	✓	71.3	64.3
Qwen3-Coder-480B-A35B-Instruct	×	59.4	44.3
	✓	69.6	54.7
SWE-HERO-32B (Ours)	×	57.7	42.2
	✓	62.2	44.1

Key: strong models have internal "world model" of code semantics

> 关键：强模型内部拥有代码语义的“世界模型”

- SWE-Zero: 300K agent trajectories that don't require repository-specific execution

> SWE-Zero：300K 条不需要仓库级执行的 agent 轨迹

- 150K GitHub PRs

> 150K 个 GitHub PR

- OpenHands scaffold, remove future git commits to prevent "git hacking" by agent

> OpenHands 脚手架，移除未来的 git 提交以防 agent 进行“git hacking”

Standard OpenHands Setup (Execution-Based)

System: You are OpenHands agent, a helpful AI assistant that can interact with a computer to solve tasks. The development Python environment is already set up for you.

Problem Solving Workflow:

- **Exploration:** Thoroughly explore relevant files and context.
- **Analysis:** Consider multiple approaches and select the most promising one.
- **Testing:** Create tests to verify issues; delete temporary files after confirmation.
- **Implementation:** Make focused, minimal changes to address the problem.
- **Verification:** Thoroughly test implementation, including edge cases.

User Instruction Phases:
Phase 1: Reading → Phase 2: Running → Phase 3: Exploration → Phase 4: Test Creation → Phase 5: Fix Analysis → Phase 6: Implementation → Phase 7: Verification → Phase 8: Final Review.

SWE-Zero OpenHands Setup (Execution-Free)

System: You are OpenHands agent... The development environment is **unavailable**. You **CANNOT RUN PYTHON CODE** for any purpose. Do not use Python to check your work.

Problem Solving Workflow:

- **Exploration:** Thoroughly explore files and understand context.
- **Analysis:** Consider multiple approaches (Static Analysis).
- **Implementation:** Make focused, minimal changes. **Do not write or execute any tests.**

Prohibited Bash Commands:
`python` (including `-c`/`-m` options), `pytest`, `mypy`, `pip`, `apt`, `apt-get`.

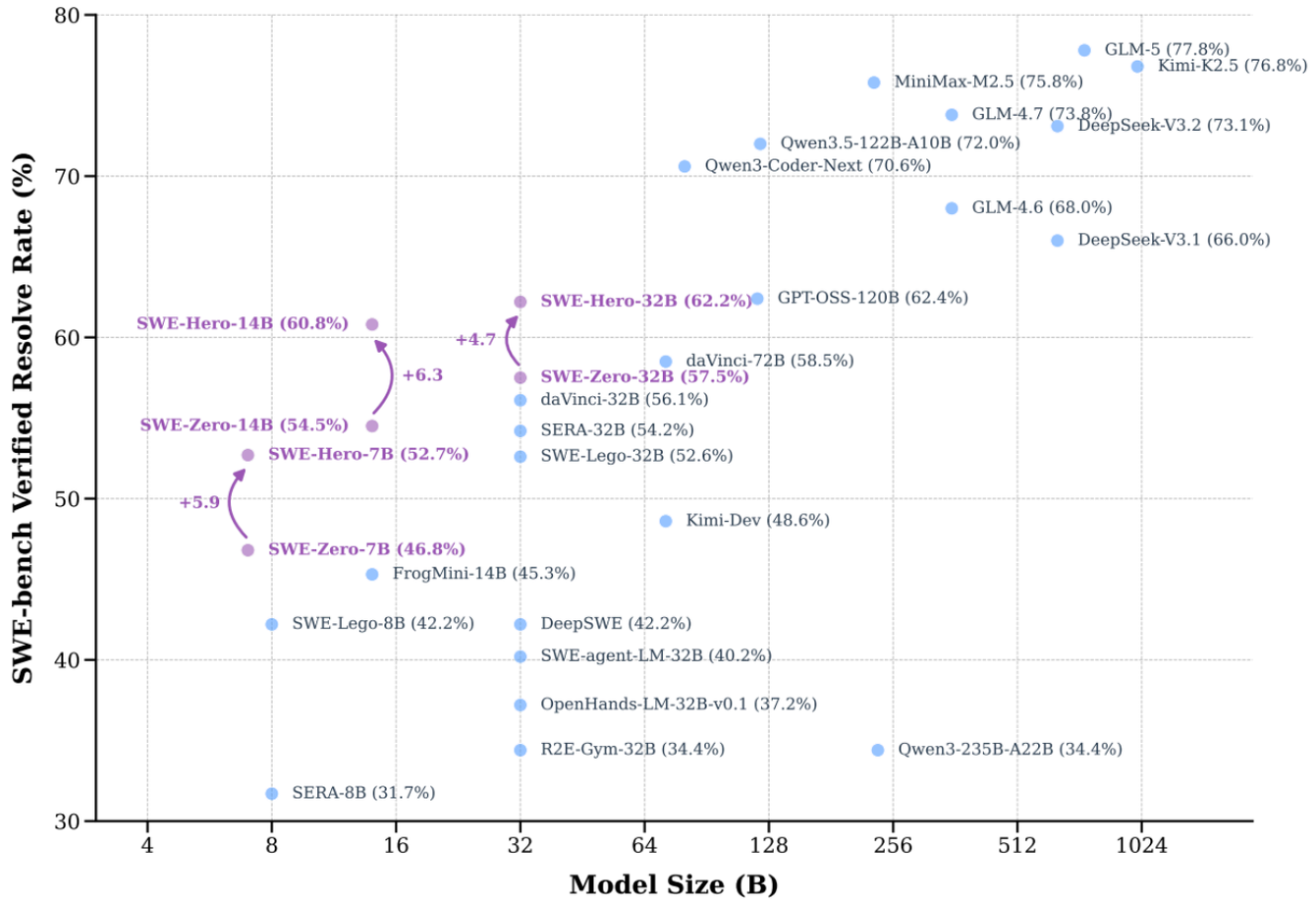
User Instruction Phases:
Phase 1: Reading → Phase 2: Exploration → Phase 3: Fix Analysis → Phase 4: Implementation → Phase 5: Final Review.

- Distilled from Qwen3-Coder-480B + filtering (try to execute anyway)

> 从 Qwen3-Coder-480B 蒸馏 + 过滤（无论如何仍会尝试执行）

- SWE-Hero: 13K agent trajectories that do require execution feedback

> SWE-Hero：13K 条确实需要执行反馈的 agent 轨迹



SWE-rebench [Badertdinov+ 2025]

- 21K interactive Python SWE tasks from 3.4K GitHub repositories

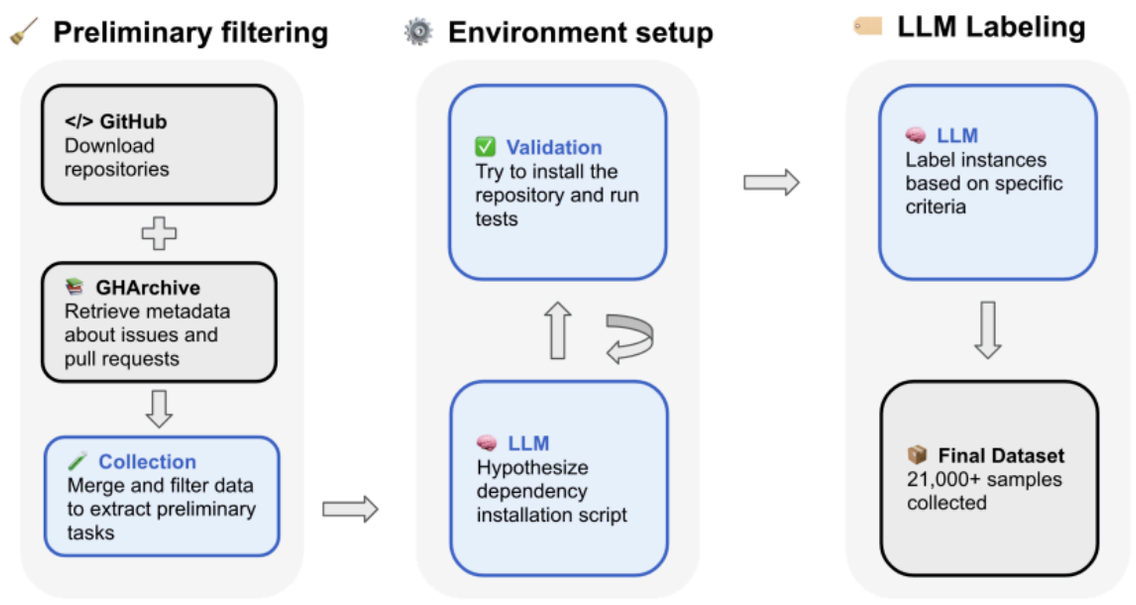
> 来自 3.4K 个 GitHub 仓库的 21K 个交互式 Python SWE 任务

- 450K PRs from GitHub and GitHub Archive

> 来自 GitHub 和 GitHub Archive 的 450K 个 PR

- Used Qwen 2.5-72B-Instruct to install dependencies and assess PR quality

> 用 Qwen 2.5-72B-Instruct 安装依赖并评估 PR 质量



SWE-ZERO-12M-trajectories [data]

> SWE-ZERO-12M-trajectories [数据]

- Scale SWE-Zero up to 12M agent trajectories

> 把 SWE-Zero 扩展到 12M 条 agent 轨迹

- Used the SWE-rebench-v2 tasks (32K executable tasks + 120K nonexecutable tasks)

> 使用 SWE-rebench-v2 任务（32K 个可执行任务 + 120K 个不可执行任务）

- Ran mini-coder-1.7b (very small model, 50.4 pass@100), mini-swe-agent scaffold

> 运行了 mini-coder-1.7b（极小的模型，50.4 pass@100）、mini-swe-agent 脚手架

- Example

> 示例

Summary:

> 小结：

- Generating prompts: fully-synthetic, semi-synthetic (real environment + synthetic tasks), real (GitHub PRs)

> 生成提示：全合成、半合成（真实环境 + 合成任务）、真实（GitHub PR）

- Responses: from capable models (that are also good teachers)

> 回答：来自有能力的模型（同时也是好教师）

- Code environments are painful

> 代码环境非常麻烦

- Lots of filtering and other details

> 大量过滤以及其他许多细节